Quest 10: The Phantasmagorithm

Lord of the Wings · ghosts that bounce off walls and each other

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: ghosts move one square per round on a cave map, turning clockwise at walls and turning around when they end a round on the same square; count their collisions.
Input: a map of `#` walls, `.` floor and ghosts `> < ^ v`.
Output: Part I: the total number of collisions in 10 rounds. Parts II and III: the round in which the last ghost disappears.
What changes between parts: Part I just counts for 10 rounds. Part II gives every ghost a collision limit (10000) and removes it when it reaches the limit. Part III uses the same rules with a limit of 10^{12}, so you can't simulate every round any more.

1. The story

While the Ents attack Nulldor from the front as a distraction, the team slips into old caves that lead under the fortress. The tunnels are guarded by Nullghosts, which drift around, turn when they hit rock and bounce off each other. Every collision makes a ghost fade a little, so the team first counts the collisions and then works out when the last ghost in each chamber will have faded away.

2. Understanding the question

The rules

```
1. MOVE   every ghost tries one step in its direction
           wall ahead?  stay put, turn 90° clockwise, +1 hit
           otherwise    step forward
2. GROUP  look at where everybody ended up
           2+ ghosts on one square?  each turns 180°, +1 hit each
3. (Part II/III) remove ghosts whose hits reached the limit
           — once after step 1, and again after step 2
```

- All ghosts move **at the same time**. Ghosts never block each other; only `#` blocks.
- Clockwise turn at a wall: `>` → `v` → `<` → `^` → `>`. The ghost does not move that round. Each ghost that bumps into a wall is one collision.
- Ghost meetings are checked **only after everyone has moved**. If $k \geq 2$ ghosts share a square, each turns around once (180°), and that's k collisions, not one per pair. A ghost can hit a wall and then also be in a meeting in the same round: 2 hits for it that round.
- Two ghosts that **swap** neighbouring squares pass through each other: they never share a square, so no collision.
- In the map pictures, a digit means "this many ghosts on this square".

The sample, after round 1 and after round 5



Round 1: the two `v` ghosts at the far left and right face a wall below them, so they turn to `<` (2 collisions). The `>` at (4,4) and the `<` at (4,6) both step onto (4,5), shown as `2`, and each turns around (2 more). Total 4. (Positions are (row, column), counting from 0.)

Extra examples of my own (run through a copy of your Part I code)

These are the puzzle's three small examples with a wall border added, because your code needs one (see the warning in Part I). A "move" here is one round of the code. The puzzle labels the *starting* picture "Round 1" in these small examples, but labels the first *result* "Round: 1" in the main example.



Map	Collisions in moves 1, 2, 3, 4	What happened
wall pair	0, 2, 0, 2	Both step next to the middle wall (move 1), bump it and turn (move 2: `v` and `^`), walk away (move 3), then bump the outer walls (move 4).
three meet	3, 0, 1, 0	All three land on (2,4) in move 1: 3 collisions and all turn around. In move 3 the one going up bumps the top wall.
swap	0, 0, 0, 0	Move 1 brings them next to each other, move 2 swaps them, and they never share a square.

Shared code: one move of one ghost

All three parts use exactly the same `opposite` table and `getCoordinate` function.

```ts
const opposite: Record<string, string> = {
  ">": "<",
  "<": ">",
  "^": "v",
  v: "^",
};

type Coordinate = { row: number; col: number; direction: string };
let COORDINATES: Coordinate[] = [];

const getCoordinate = (
  inputs: string[][],
  coordinate: Coordinate,
): {
  coordinate: Coordinate;
  collisions: number;
} => {
  let collisions = 0;

  let row = coordinate.row;
  let col = coordinate.col;
  let direction = coordinate.direction;

  switch (direction) {
    case ">":
      if (inputs[row]![col + 1] === "#") {
        direction = "v";
        collisions++;
      } else col++;
      break;
    case "<":
      if (inputs[row]![col - 1] === "#") {
        direction = "^";
        collisions++;
      } else col--;
      break;
    case "^":
      if (inputs[row - 1]![col] === "#") {
        direction = ">";
        collisions++;
      } else row--;
      break;
    case "v":
      if (inputs[row + 1]![col] === "#") {
        direction = "<";
        collisions++;
      } else row++;
      break;
  }

  return {
    coordinate: {
      row,
      col,
      direction,
    },
    collisions,
  };
```

```
68 };
```

q10/part1.ts, lines 10–68

Lines	What happens
10–15	`opposite`: the 180° turn for a ghost meeting. Parts II and III also use its keys to recognise a ghost (`cell in opposite`).
17	A ghost is its row, column and direction.
20–31	`getCoordinate` works on a copy of the position and starts with 0 collisions.
33–58	One case per direction: look at the square ahead. If it's `#`, turn clockwise and count one collision; otherwise step. The four `direction = …` lines are the clockwise rule written out by hand.
60–67	Return the new ghost and how many wall hits (0 or 1) it had.

3. Part I: count collisions for 10 rounds

The question

Run 10 rounds and add up all wall collisions and all ghost-meeting collisions.

Worked example

Round	1	2	3	4	5	6	7	8	9	10	Total
Collisions	4	3	1	1	8	3	1	3	1	4	**29**

Your code prints exactly these per-round numbers (checked with an instrumented copy) and the total 29.

Step-by-step solution

1. Read the map as a grid of characters.
2. Each round, scan the grid. Pick up every ghost (for a digit square, get the ghosts from last round's list), move it one step or turn it clockwise at a wall, and count wall hits.
3. Put all moved ghosts back. A square that gets a second ghost becomes `2` (+2 collisions); every further ghost adds 1.
4. Turn every ghost on a digit square around.
5. Repeat for 10 rounds and add up the collisions.

My solution

Part I keeps the ghosts **on the map itself**: each round it picks every ghost up off the grid, moves it, and writes it back. Squares with several ghosts get a digit, just like the puzzle's pictures.

```
const getCollisions = (inputs: string[][]): number => {
  let totalCollisions = 0;
  let coordinates = [];

  for (let row = 0; row < inputs.length; row++) {
    for (let col = 0; col < inputs[0]!.length; col++) {
      if (inputs[row]![col] === "#" || inputs[row]![col] === ".") continue;

      let direction = inputs[row]![col] || "";

      if (!isNaN(Number(direction))) {
        const foundCoordinates = COORDINATES.filter(
          (c) => c.row === row && c.col === col,
        );

        foundCoordinates.forEach((c) => {
          const { coordinate, collisions } = getCoordinate(inputs, c);

          coordinates.push(coordinate);
          totalCollisions += collisions;
        });

        inputs[row]![col] = ".";
        continue;
      }

      const { coordinate, collisions } = getCoordinate(inputs, {
        row,
        col,
        direction,
      });

      coordinates.push(coordinate);
      totalCollisions += collisions;

      inputs[row]![col] = ".";
    }
  }

  coordinates.forEach(({ row, col, direction }) => {
    if (inputs[row]![col] !== ".") {
      if (!isNaN(Number(inputs[row]![col]))) {
        totalCollisions++;
        inputs[row]![col] = String(Number(inputs[row]![col]) + 1);
      } else {
        totalCollisions += 2;
        inputs[row]![col] = "2";
      }
    } else inputs[row]![col] = direction;
  });

  coordinates.forEach((c) => {
    if (!isNaN(Number(inputs[c.row]![c.col])))
      c.direction = opposite[c.direction]!;
  });

  COORDINATES = coordinates;

```

```
  return totalCollisions;
};

let totalCollisions = 0;

for (let i = 1; i <= 10; i++) {
  totalCollisions += getCollisions(INPUTS);
}

console.log(totalCollisions);
```

q10/part1.ts, lines 70–137

Lines	What happens
18	`COORDINATES` (global): the list of ghosts from the previous round. It's only needed to find out which ghosts are hidden behind a digit, because a digit doesn't say which directions they face.
74–78	Scan the grid row by row. Skip walls and floor; anything else is a ghost (or a digit).
80–94	A digit: take all ghosts from last round's list that stand on this square, move each one, and clear the square.
96–105	An arrow: move that ghost and clear its old square. The new position is **not** written yet (it goes into `coordinates`), so a ghost that moves right or down can't be found and moved again later in the same scan.
109–119	Write everyone back. Empty square: draw the arrow. Square already has an arrow: this is the second ghost, so it becomes `2` and both ghosts count (+2). Square already has a digit: one more ghost, +1.
121–124	Every ghost standing on a digit turns around.
126–128	Remember this round's list for the next round, and return the collisions.
131–137	Ten rounds, add up, print.

Trace: what the code holds after each of the first rounds (sample)

Round	Collisions	`COORDINATES` after the round (row,col + direction)
1	4	4,4> 4,5< 4,5> 4,6< 5,1< 5,6> 5,9< 7,5v
2	3	4,5< 4,4< 4,6> 4,5> 5,1^ 5,7> 5,8< 8,5v
3	1	4,3< 4,4< 4,6> 4,7> 5,1> 5,8> 5,7< 9,5v
4	1	4,2< 4,3< 4,7> 4,8> 5,2> 5,6< 5,9> 9,5<

After round 1 two ghosts stand on (4,5), already turned around (`4,5<` and `4,5>`), and the grid holds a `2` there. In round 2 those two walk apart to (4,4) and (4,6), while the ghosts that started at (4,3) and (4,7) walk into (4,5) from both sides, so (4,5) is a `2` again. That is the "another two finished on the same space" in the puzzle's round 2. Round 2's wall hit is the left ghost at (5,1) turning from `<` to `^`.

Fragile 1: the map must have a wall all the way round. `getCoordinate` reads `inputs[row - 1][col]` with no bounds check. On the puzzle's small example `>.#.<` written without a border, a copy of your code ran 3 rounds and then crashed in round 4 with *TypeError: Cannot read properties of undefined*, when a ghost on the top row tried to look above it. Real chambers are always closed by `#`, so it doesn't matter for your input. The same function is in Parts II and III.

Fragile 2: two copies of the truth. The ghosts live on the grid *and* in `COORDINATES`, and the digit squares only work because the two agree. It's correct, but Part II's list-only design (no ghosts on the grid at all) is much simpler, and would also have worked here.

4. Part II: when does the last ghost vanish?

The question

Same rules, but each ghost disappears right after its 10000th collision (10 in the example). The order matters: a ghost that dies from a wall hit is gone **before** meetings are checked, so it can't cause a meeting. Report the round in which the last ghost disappears.

Worked examples

Limit	Rounds where the count drops (ghosts left)	Last
10	15 (7), 23 (5), 24 (4), 27 (3), 29 (2), 48 (1), 53 (0)	**53**
10000	10010 (7), 10012 (6), 10025 (5), 10045 (4), 33299 (3), 33349 (2), 49977 (1), 50000 (0)	**50000**

Both rows come from a copy of your Part II that logs every drop. They match the puzzle exactly.

An extra example of my own: the wall-death rule

```
####
#>>#
####
```

limit 1

With a limit of 1: in round 1 the right ghost bumps the wall (1 hit, gone) and the left ghost steps onto its square. Because the dead ghost is removed first, there is no meeting; the left ghost dies in round 2 when it bumps the wall. Answer **2**. If you remove the filter on line 96, the two "meet" in round 1, both get a hit and the answer becomes **1** (tested with a copy). On the real sample the filter changes nothing (53 and 50000 either way), so the sample doesn't test this rule.

Step-by-step solution

1. Take every ghost off the map into a list, with a hit counter of 0.
2. Each round: move every ghost (adding wall hits), then remove ghosts that reached the limit.
3. Group the survivors by square; in each group of 2 or more, turn each ghost around and add one hit. Remove the dead again.
4. Stop when the list is empty; the round number is the answer.

My solution

Lines 1–9 are the same reading code (with `sample2.txt` / `input2.txt`), and 12–70 are the shared `opposite` / `getCoordinate`.

```
10  const MAX_COLLISION = isSample ? 10 : 10000;
11
12  const opposite: Record<string, string> = {
13    ">": "<",
14    "<": ">",
15    "^": "v",
16    v: "^",
17  };
18
19  type Coordinate = { row: number; col: number; direction: string };
20  type Ghost = Coordinate & { hits: number };
```

q10/part2.ts, lines 10–20 (highlighted lines are the ones that changed from the previous part)

```ts
let ghosts: Ghost[] = [];

for (let row = 0; row < INPUTS.length; row++) {
  for (let col = 0; col < INPUTS[row]!.length; col++) {
    const cell = INPUTS[row]![col]!;
    if (cell in opposite) {
      ghosts.push({ row, col, direction: cell, hits: 0 });
      INPUTS[row]![col] = ".";
    }
  }
}

const isAlive = (g: Ghost) => g.hits < MAX_COLLISION;

let round = 0;

while (ghosts.length > 0) {
  round++;

  ghosts = ghosts.map((g) => {
    const { coordinate, collisions } = getCoordinate(INPUTS, g);
    return { ...coordinate, hits: g.hits + collisions };
  });

  ghosts = ghosts.filter(isAlive);

  const groups = new Map<string, Ghost[]>();

  for (const g of ghosts) {
    const key = `${g.row},${g.col}`;
    groups.set(key, [...(groups.get(key) ?? []), g]);
  }

  for (const group of groups.values()) {
    if (group.length < 2) continue;

    for (const g of group) {
      g.direction = opposite[g.direction]!;
      g.hits++;
    }
  }

  ghosts = ghosts.filter(isAlive);
}

console.log(round);
```

q10/part2.ts, lines 72–117 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
10	The limit: 10 for the sample, 10000 for the real input.
20	A ghost now also carries `hits`, its collision count.
72–82	Take every ghost off the map once, at the start. From now on the grid only holds walls and floor.
84	Alive means fewer than `MAX_COLLISION` hits.
88–89	Keep doing rounds until no ghost is left. `round` is then the round of the last death.
91–94	Move everyone and add the wall hits.
96	Remove ghosts killed by a wall **before** grouping. This is the rule from the extra example above.
98–103	Group the survivors by square, with a `Map` from `"row,col"` to the ghosts there.
105–112	Every group of 2 or more: each ghost turns around and gets one hit, however big the group is.
114	Remove ghosts killed by a meeting.

Trace: hits per ghost in the first 5 rounds (sample, from an instrumented copy)

Ghost	Start	r1	r2	r3	r4	r5
#0	4,3 >	0	1	1	1	2
#1	4,4 >	1	1	1	1	3
#2	4,6 <	1	1	1	1	3
#3	4,7 <	0	1	1	1	2
#4	5,1 v	1	2	3	3	3
#5	5,5 >	0	0	0	0	1
#6	5,9 v	1	1	1	1	1
#7	6,5 v	0	0	0	1	2
sum		4	7	8	9	17

The sums are the running totals of Part I (4, 4+3, +1, +1, +8). In round 5 ghost #1 gets 2 hits: it stands at (4,2) facing `<` with a wall on its left, so it turns clockwise to `^` (hit 2). Meanwhile ghost #0 steps from (4,3) onto the same square, so both turn around: #1 ends up facing `v` (hit 3) and #0 faces `>`.

> **Cost:** one step per round, and each step touches every ghost. With a limit of 10000 the sample needs 50000 rounds, and your real input runs in about 0.14 s. But the number of rounds grows with the limit, so with Part III's 10^{12} this would take trillions of rounds.
> **Small thing:** line 102 copies the whole group array each time a ghost is added (`[...old, g]`), which is quadratic in the group size. Groups are tiny here, so it doesn't matter.

5. Part III: a limit of a trillion

The question

The same rules as Part II, but each ghost survives 10^{12} collisions. The example chamber has 36 ghosts and the last one disappears in round 13314695352588 (your code gives exactly this).

The key idea: the ghosts' movement repeats

Where the ghosts go depends only on their positions and directions, **not** on their hit counts (hits only matter when someone dies). The map is finite, so between two deaths the positions and directions must eventually come back to a state seen before, and from then on everything repeats with some **period** p. In each period every ghost gains a fixed number of hits d. So you can skip many periods at once: add $c \cdot p$ rounds and $c \cdot d$ hits, as long as nobody dies during the skip. This is **cycle detection** (like the "find the loop, then jump ahead" trick in many simulation puzzles).

Step-by-step solution

1. Run Part II's round as a function that says whether anybody died.
2. After each round with no death, record the state (positions and directions). When a state repeats, the difference in rounds is the period.
3. Simulate one more period to measure each ghost's hits per period.
4. Skip as many whole periods as possible while every ghost stays below the limit, then keep simulating normally until the next death.
5. Reset the state record after every death or skip, and repeat until no ghost is left.

My solution

Lines 1–83 are Part II's code, except line 10 (the limit is always 10^{12}).

```ts
const isAlive = (g: Ghost) => g.hits < MAX_COLLISION;

let round = 0;

const step = (): boolean => {
  round++;
  const before = ghosts.length;

  ghosts = ghosts.map((g) => {
    const { coordinate, collisions } = getCoordinate(INPUTS, g);
    return { ...coordinate, hits: g.hits + collisions };
  });

  ghosts = ghosts.filter(isAlive);

  const groups = new Map<string, Ghost[]>();

  for (const g of ghosts) {
    const key = `${g.row},${g.col}`;
    groups.set(key, [...(groups.get(key) ?? []), g]);
  }

  for (const group of groups.values()) {
    if (group.length < 2) continue;

    for (const g of group) {
      g.direction = opposite[g.direction]!;
      g.hits++;
    }
  }

  ghosts = ghosts.filter(isAlive);

  return ghosts.length !== before;
};

const stateKey = () =>
  ghosts.map((g) => `${g.row},${g.col},${g.direction}`).join("|");

let seen = new Map<string, number>();

while (ghosts.length > 0) {
  if (step()) {
    seen = new Map();
    continue;
  }

  const key = stateKey();
  const firstSeen = seen.get(key);

  if (firstSeen === undefined) {
    seen.set(key, round);
    continue;
  }

  const period = round - firstSeen;
  const hitsBefore = ghosts.map((g) => g.hits);

```

```
  let died = false;

  for (let i = 0; i < period && !died; i++) died = step();

  seen = new Map();
  if (died) continue;

  const delta = ghosts.map((g, i) => g.hits - hitsBefore[i]!);

  let cycles = Infinity;

  ghosts.forEach((g, i) => {
    if (delta[i]! > 0)
      cycles = Math.min(
        cycles,
        Math.floor((MAX_COLLISION - 1 - g.hits) / delta[i]!),
      );
  });

  if (cycles === Infinity) throw new Error("ghosts never take damage");

  round += cycles * period;
  ghosts.forEach((g, i) => (g.hits += cycles * delta[i]!));
}

console.log(round);
```

q10/part3.ts, lines 84–167 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
88–118	`step()` is Part II's loop body as a function. It returns `true` if any ghost died in this round.
120–121	`stateKey`: every ghost's position and direction, in list order. Hits are left out on purpose: they don't affect the movement.
123	`seen`: state → the round when it was first seen.
126–129	Someone died: the old states had more ghosts, so they can never repeat. Start the search again.
131–137	A new state: remember it and go on.
139	A repeat. The period is the distance between the two rounds.
140–147	Simulate one more full period and see how many hits each ghost gets in it. If anybody dies during this, give up and keep searching.
149	`delta[i]`: hits per period of ghost *i*. The list order is the same as before because nobody died.
151–159	How many whole periods can we skip? Ghost *i* must still be alive afterwards: hits + c · delta ≤ MAX − 1, so c = ⌊(MAX − 1 − hits) / delta⌋. Take the smallest over all ghosts.
161	Safety check: if no ghost ever got hurt, the loop would run forever.
163–164	Jump: move the clock and everybody's hits forward. Positions stay the same, since we skipped whole periods.
125	Back to normal steps. The ghost that limited *c* now has less than one period of hits left, so it dies within the next period.

Trace: every skip on the sample (from an instrumented copy)

Ghosts	Repeat at round	first seen	Period	Periods skipped	Clock after the jump
36	810	306	504	1322751320	666666666594
34	666666667301	666666666797	504	661375659	999999999941
11	1000000003129	1000000002409	720	617283945	1444444444249
9	1444444445694	1444444445470	224	566893427	1571428573566
8	1571428573945	1571428573721	224	645628613	1716049383481
7	1716049383886	1716049383662	224	110229274	1740740741486
5	1740740743580	1740740741812	1768	52371374	1833333334580
4	1833333344412	1833333336748	7664	48904536	2208137715980
3	2208137719896	2208137718128	1768	1794677125	5381126878664
2	5381126880893	5381126880053	840	7029879709	11286225837293
1	11286225837632	11286225837576	56	36222669908	13314695352536

Eleven jumps, and only 39892 rounds are actually simulated to reach round 13314695352588. Compare the rows with the puzzle's table: after the first jump to 666666666594, the next deaths come at 666666666760 and 666666666796, both within one period (504 rounds), just as the formula promises. Between 999999999941 and 1000000003129 there is no jump at all: 22 ghosts die between rounds 999999999994 and 1000000000236, and every death resets the state search, so no repeat is found until a while after the last of them. Your real input needs 10 jumps and about 44000 simulated rounds.

A cross-check: running the Part III logic on the Part II sample gives **53** (limit 10, no jump needed) and **50000** (limit 10000, 300 rounds simulated), the Part II answers. One of those jumps has $c = 0$: a ghost was already within one period of its limit, so the measured period was wasted, but harmless.

> **Why the skip is safe:** hits only ever go up, so if every ghost is still alive at the end of the jump (hits ≤ MAX − 1), it was alive at every round in between. And the state after whole periods is the same state, so the jump lands exactly where a normal simulation would have been.
>
> **Numbers:** the largest round, about 1.3×10^{13}, is far below $2^{53} \approx 9.0 \times 10^{15}$, so normal JavaScript numbers are exact here. No `bigint` needed.
>
> **The state key keeps ghost identity:** two ghosts that swap places give a different key, even though the map looks the same. That's what you want, because their hit counts differ. At worst it makes a period look longer than the picture's period.

6. Summary

Part	Asks for	How the ghosts are stored	Rounds simulated (sample)	Sample answer
I	collisions in 10 rounds	on the grid, plus last round's list for digit squares	10	29
II	round of the last death, limit 10 / 10000	list of ghosts with `hits`	53 / 50000	53 / 50000
III	the same, limit 10^{12}	same list, plus a state → round map	39892	13314695352588

Key ideas: a simultaneous-move simulation is easiest with a **list** of movers and a "group by square" step after everyone has moved. When the limit is huge, notice that the state (without the counters) **repeats**, measure what one period adds, and **jump** as many periods as you safely can.